

Backtester Module

Sports Betting Strategy Evaluation Engine

Presenter: James
Date: 25 February 2026
Branch: james_branch → main
PR: #53
Tests: 57/57 passing

Cardiff University Investment Club
Quant Team — Semester 2

Contents

1	What I Built	2
1.1	Core Functions	2
1.2	Example Strategies	2
2	The Pipeline	2
2.1	9-Column Output Format	2
3	Key Features	2
3.1	Data Leakage Prevention	2
3.2	Transaction Cost Modeling	3
3.3	Kelly Criterion Position Sizing	3
3.4	Validation Suite (11 Checks)	3
4	Demo Results	4
5	PR Review Fixes	4
5.1	Critical Fixes (C1–C5)	4
5.2	Important Fixes (I1–I8)	5
5.3	Test Gaps Filled	5
6	Files Changed	5
7	What’s Next (Roadmap)	5
8	How to Run	6

What I Built

A complete backtesting engine for evaluating sports betting strategies against historical data, implemented as an importable Python module at:

`src/cuic_quant/backtest/backtester_backend.py`

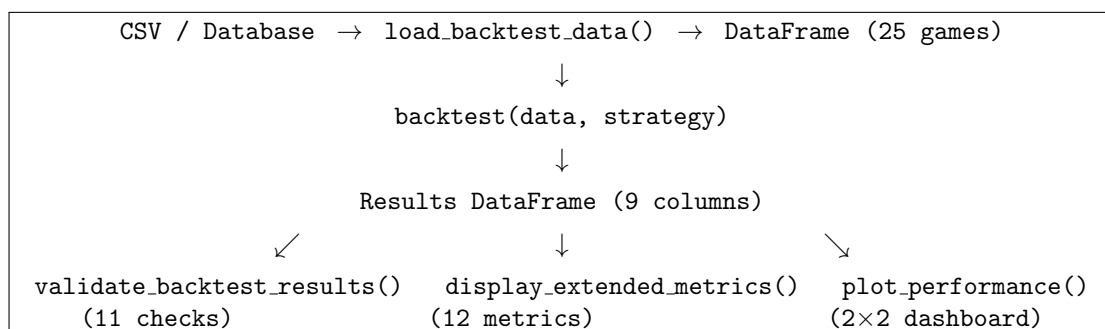
Core Functions

Function	Purpose
<code>load_backtest_data()</code>	Loads historical games from Railway DB or local CSV
<code>backtest()</code>	Runs a strategy over historical data, tracks PnL and bankroll
<code>validate_backtest_results()</code>	11-check validation suite (schema, math, data leakage)
<code>display_extended_metrics()</code>	12-metric performance table (Sharpe, Sortino, draw-down, etc.)
<code>plot_performance()</code>	2×2 matplotlib dashboard

Example Strategies

Strategy	Behaviour
<code>always_bet_home()</code>	Fixed \$100 on home team every game — deterministic test strategy
<code>always_bet_away()</code>	Fixed \$100 on away team — tests BUY_AWAY logic path
<code>kelly_bet_home_demo()</code>	Kelly-sized home bets — demo only , shows plumbing

The Pipeline



9-Column Output Format

Every strategy produces the same output — this is the contract that Ben’s metrics and Ismaeel’s tests depend on:

Key Features

Data Leakage Prevention

The backtester **strips** `home_win` from the row before passing it to the strategy function. Strategies cannot cheat by looking at the outcome. A dedicated “spy strategy” test verifies no

Column	Type	Example
timestamp	datetime	2026-01-01
game	str	Lakers vs Celtics
action	str	BUY_HOME
bet_size	float	256.58
odds	float	1.95
outcome	str	WIN
pnl	float	243.75
cumulative_pnl	float	243.75
bankroll	float	10243.75

strategy ever receives `home_win`.

Transaction Cost Modeling

Two cost parameters model real-world betting friction:

- `cost_pct` — Percentage deducted from winning payouts (models bookmaker vig)
- `cost_flat` — Flat dollar fee per trade regardless of outcome

PnL formulas:

WIN: `bet_size × (odds - 1) × (1 - cost_pct) - cost_flat`
 LOSS: `- bet_size - cost_flat`

Kelly Criterion Position Sizing

Instead of flat \$100 bets, Kelly sizes each bet as a fraction of bankroll based on estimated edge:

$$f^* = \frac{bp - q}{b}$$

Variable	Meaning	Example (odds = 1.95)
f^*	Fraction of bankroll to bet	0.1026 ($\approx 10.3\%$)
b	Net odds = decimal_odds - 1	$1.95 - 1 = 0.95$
p	Estimated win probability	0.5628
q	Loss probability = $1 - p$	0.4372

Kelly Fraction	Name	Risk Profile
0.25	Quarter Kelly	Very conservative
0.50	Half Kelly	Moderate (default)
0.75	Three-quarter Kelly	Aggressive
1.00	Full Kelly	Maximum growth (unrealistic)

Validation Suite (11 Checks)

- **Schema (5 checks):** Correct columns, valid actions/outcomes, positive bets, valid odds
- **Math (4 checks):** PnL formula correctness, cumulative PnL running sum, bankroll tracking, no overbetting
- **Data Leakage (2 checks):** Outcomes match input data, chronological order

Demo Results

Running `kelly_bet_home_demo` with quarter-Kelly on 25 NBA games:

Metric	Value
Win Rate	60.0% (15W / 10L)
Total PnL	\$1,123.02
Final Bankroll	\$11,123.02
ROI	11.2%
Sharpe Ratio	2.808
Max Drawdown	4.8%
Profit Factor	1.434
Validation	12/12 checks passed

PR Review Fixes

The first PR was reviewed by Tan and returned with feedback across three categories. **All items have been addressed.**

Critical Fixes (C1–C5)

C1. `except Exception`: silently swallowed **ALL** errors

- **Problem:** `__init__.py` caught `SyntaxError`, `AttributeError`, `RecursionError` — not just `ImportError`.
- **Fix:** Changed to `except ImportError`: with `logging.debug()`.

C2. `kelly_bet_home` had circular edge logic

- **Problem:** Confidence = implied_prob + 5%. Since Kelly is positive whenever $p > 1/\text{odds}$, and here $p = 1/\text{odds} + 0.05$, the strategy always bets. Tautological.
- **Fix:** Renamed to `kelly_bet_home_demo` with a prominent `WARNING` docstring.

C3. Sharpe/Sortino ratios were mathematically meaningless

- **Problem:** (1) Used raw dollar PnL, not % returns. (2) Annualized with $\sqrt{252}$ (trading days). (3) Sortino used `std(losses_only)`.
- **Fix:** (1) Now uses `pnl / bankroll_before_bet`. (2) Annualizes with $\sqrt{365}$. (3) New `calculate_sortino_ratio()` with correct downside deviation:

$$\sigma_d = \sqrt{\frac{1}{N} \sum_{i=1}^N \min(0, r_i)^2}$$

C4. Bankroll could go negative with `cost_flat`

- **Problem:** `bet_size` capped at `bankroll`, but `loss = -(bet_size + cost_flat)` could exceed `bankroll`.
- **Fix:** `bet_size = min(bet_size, max(0, bankroll - cost_flat))`

C5. Database fallback silently loaded dummy CSV

- **Problem:** When `DATABASE_URL` was set but query failed, code `print()`'d a message and loaded synthetic data.
- **Fix:** `warnings.warn(..., RuntimeWarning)`. Added `strict=True` mode that raises.

Important Fixes (I1–I8)

Fix	What Changed
I1	Kelly import moved from inside for-loop to top of function
I2	Warning when Kelly falls back to flat sizing (confidence outside (0, 1))
I3	Input column validation — raises <code>ValueError</code> for missing columns
I4	<code>try/except ImportError</code> fallback in <code>display_extended_metrics()</code>
I5	Warning on unrecognized strategy actions (e.g. “BUY_DRAW”)
I6	Google-style docstring added to <code>always_bet_away()</code>
I7	Cost params stored in <code>DataFrame.attrs</code> metadata
I8	matplotlib import moved inside <code>plot_performance()</code>

Test Gaps Filled

Test	What It Verifies
Bankroll negative with <code>cost_flat</code>	Bankroll ≥ 0 at all times with flat fees (validates C4)
Multi-trade PnL with costs <code>always_bet_away</code> dedicated test	Row-by-row PnL verification across 5 trades Action, context handling, docstring existence
Kelly confidence boundaries	0.0 and 1.0 correctly fall back to flat sizing
Empty <code>DataFrame</code> input	Returns empty results with correct 9 columns
All-rows bankroll check	Every row has bankroll ≥ 0 , not just the last
Missing columns	Raises <code>ValueError</code> (validates I3)

Test suite: 57 tests, all passing.

Files Changed

File	Lines	Role
<code>src/cuic_quant/__init__.py</code>	50	Package init
<code>src/cuic_quant/backtest/backtester_backend.py</code>	934	Core backtester
<code>src/cuic_quant/backtest/__init__.py</code>	37	Package exports
<code>src/cuic_quant/metrics/__init__.py</code>	185	Performance metrics
<code>tests/test_backtester_backend.py</code>	999	57 tests

What’s Next (Roadmap)

Items flagged in review as “Domain Fitness” — not blocking merge but tracked for future work:

- Walk-forward validation / train–test split for real strategy evaluation
- Closing Line Value (CLV) — the #1 metric for sports betting skill
- Polymarket/Kalshi compatibility (CLOB / YES–NO contracts)
- Multi-bookmaker odds (OddsAPIClient already supports this)

- Point spreads (currently moneyline only)
- Expose Kelly `max_fraction=0.25` cap in `backtest()` parameters

How to Run

```
# Run the demo notebook
jupyter lab tools/backtester.ipynb

# Run tests
pytest tests/test_backtester_backend.py -v

# Use in your own code
from cuic_quant.backtest import backtest, load_backtest_data

data = load_backtest_data("2026-01-01", "2026-01-31")
results = backtest(data, your_strategy, position_sizing="kelly")
```